

NEUROTRAFFIC: SMART TRAFFIC CONTROL SYSTEM

AI-Driven Adaptive Traffic Management

*A Mini Project Report submitted in partial fulfillment of the requirements for the
award of the degree of*

Bachelor of Technology
in
Computer Science and Engineering

by

Adith M (TKR23CS011)
Anal KP (TKR23CS022)
Anju Karun (TKR23CS026)
Anukheth Sunil (TKR23CS028)

Department of Computer Science and Engineering
College of Engineering Trikaripur
APJ Abdul Kalam Technological University

DECLARATION

We, the undersigned, hereby declare that the mini project report entitled "**NeuroTraffic: Smart Traffic Control System**", submitted to the Department of Computer Science and Engineering, College of Engineering Trikaripur, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in Computer Science and Engineering, is a record of original work carried out by us under the guidance of our project coordinator.

Adith M
(TKR23CS011)

Anal KP
(TKR23CS022)

Anju Karun
(TKR23CS026)

Anukheth Sunil
(TKR23CS028)

Date:

Place: Trikaripur

CERTIFICATE

This is to certify that the mini project report entitled "**NeuroTraffic: Smart Traffic Control System**", is a bonafide record of the work carried out by **Anukheth Sunil, Anal KP, Anju Karun, and Adith M** under our supervision and guidance, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at College of Engineering Trikaripur. This report in any form has not been submitted to any other University or Institute for any purpose.

Project Coordinator

Head of Department

ACKNOWLEDGEMENT

We would like to express our deep sense of gratitude to our project guide, for their valuable guidance and constant encouragement throughout the course of this mini project. Their insights were instrumental in the successful completion of **NeuroTraffic**.

We are also thankful to our Head of Department, for providing us with the necessary facilities and support. Finally, we would like to thank our parents and friends for their continuous support and motivation.

ABSTRACT

Urbanization has led to a significant increase in vehicle density, making traditional traffic management systems inefficient. Most current traffic signals operate on fixed timers, leading to unnecessary delays, increased fuel consumption, and congestion. **NeuroTraffic** is an AI-driven adaptive traffic management system designed to optimize traffic signal timings in real-time. By utilizing YOLO26 for vehicle detection, Deep Q-Networks (DQN) for reinforcement learning-based signal control, and LSTM models for congestion prediction, the system replaces static timers with dynamic, demand-based control. Integrated with the SUMO (Simulation of Urban MObility) engine, NeuroTraffic demonstrates a significant reduction in average waiting times and queue lengths. The system also features emergency vehicle prioritization and illegal parking detection, making it a comprehensive solution for modern smart city traffic management.

Contents

Declaration	i
Certificate	ii
Acknowledgement	iii
Abstract	iv
Abbreviations	ix
1 Introduction	1
1.1 Overview	1
1.2 Motivation	1
1.3 Problem Statement	1
1.4 Objectives	2
1.5 Scope	2
1.6 Feasibility Study	2
1.6.1 Technical Feasibility	2
1.6.2 Economic Feasibility	3
1.6.3 Operational Feasibility	3
2 System Requirements	4
2.1 Existing Systems	4
2.1.1 Pre-timed Traffic Signal Control	4
2.1.2 Inductive Loop Sensors	4
2.2 Requirement Analysis	4
2.2.1 Functional Requirements	4
2.2.2 Non-Functional Requirements	5
2.3 Hardware Requirements	5
2.4 Software Requirements	5

3	System Design	6
3.1	System Architecture	6
3.2	Module Description	6
3.2.1	Processing Module	6
3.2.2	RL Control Module	6
3.2.3	Database Design	7
3.3	Data Flow Diagram	7
3.4	Entity Relationship Diagram	8
4	Implementation	9
4.1	Methodology	9
4.2	Algorithms	9
4.2.1	Perception Algorithm (YOLO26)	9
4.2.2	Decision Algorithm (RL Pipeline)	10
4.3	Development Tools	10
5	Results and Discussion	11
5.1	Simulation Snapshots	11
6	Testing and Results	12
6.1	Simulation Metrics	12
6.2	User Interface Results	12
7	Conclusion	15
7.1	Summary	15
7.2	Limitations	15
7.3	Future Work	15
A	API Documentation	17
A.1	Core Data Endpoints	17
A.2	Simulation & Control Endpoints	17
A.3	Historical Reporting Endpoints	18
A.4	WebRTC Streaming	18

List of Figures

3.1	Data Flow Diagram	7
3.2	Entity Relationship Diagram	8
6.1	Main Dashboard: Live Traffic Monitor	13
6.2	System Archive: Historical Traffic Logs	13
6.3	Simulation and Control Panel	13
6.4	Real-time Traffic Volume Trends	14

List of Tables

3.1	Database Schema: TRAFFIC_LOG	7
3.2	Database Schema: SIGNAL_SCHEDULE	7
6.1	Performance Comparison: Static vs. NeuroTraffic	12

ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
DQN	Deep Q-Network
FPS	Frames Per Second
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
LSTM	Long Short-Term Memory
RAM	Random Access Memory
REST	Representational State Transfer
RL	Reinforcement Learning
SDP	Session Description Protocol
SQL	Structured Query Language
SUMO	Simulation of Urban MObility
TraCI	Traffic Control Interface
UI	User Interface
UUID	Universally Unique Identifier
V2I	Vehicle-to-Infrastructure
V2X	Vehicle-to-Everything
WebRTC	Web Real-Time Communication
YOLO	You Only Look Once

Chapter 1

Introduction

1.1 Overview

Cities are expanding rapidly, and road infrastructure is struggling to keep pace with the exponential growth in the number of vehicles. Efficient traffic management is no longer just a convenience but a necessity for economic productivity and environmental sustainability. Traditional traffic control systems, which rely on pre-defined timing cycles, fail to account for the stochastic nature of traffic flow. NeuroTraffic aims to bridge this gap by introducing intelligence into the intersection.

1.2 Motivation

Traditional traffic lights are "blind" to the actual density of vehicles on the road. This leads to the frustrating "phantom stop" where drivers wait at a red light while the cross-lane is completely empty. Moreover, the lack of an integrated emergency priority system often results in critical delays for ambulances and fire trucks. The motivation behind **NeuroTraffic** is to harness the power of Convolutional Neural Networks (CNNs) and Reinforcement Learning to create a system that can see, predict, and react to traffic in real-time, thereby reducing carbon emissions, fuel consumption, and travel time.

1.3 Problem Statement

Traditional traffic signals follow pre-set timing cycles regardless of real-time conditions. This lack of adaptability leads to:

- **Inefficiency:** Unnecessary waiting at red lights when the cross-lane is empty.
- **Congestion surcharging:** Heavy traffic during peak hours cannot be cleared effectively by rigid timers.

- **Emergency Delays:** Significant delays in emergency vehicle movement due to lack of preemption.
- **Information Vacuum:** Lack of real-time monitoring and data-driven insights for urban planning.

1.4 Objectives

The primary goal of NeuroTraffic is to develop an AI-powered adaptive traffic signal control system. The specific objectives include:

- **Real-time Vehicle Detection:** Accurately counting and classifying vehicles using computer vision (YOLO26).
- **Predictive Analytics:** Forecasting upcoming congestion surcharges using LSTM models.
- **Reinforcement Learning Control:** Implementing a DQN agent to optimize green-light phases.
- **Emergency Prioritization:** Providing automated green corridors for detected emergency vehicles.

1.5 Scope

The scope of this project includes the development of a simulated environment (SUMO) and a real-time vision-based backend (FastAPI). It covers the detection of vehicles from overhead camera feeds, the training of an RL agent in a virtual environment, and the deployment of a web-based dashboard for monitoring. The system is currently designed for a standard 4-way intersection but can be scaled to multi-junction networks.

1.6 Feasibility Study

1.6.1 Technical Feasibility

The project leverages established technologies such as Python, PyTorch, and YOLO26. The availability of high-performance GPUs and robust simulation environments like SUMO makes it technically feasible to develop and test the proposed system.

1.6.2 Economic Feasibility

NeuroTraffic aims to reduce fuel consumption and wear and tear on vehicles by optimizing traffic flow. The cost of implementation involves the purchase of cameras and a central processing unit, which is offset by the long-term economic benefits of reduced congestion and improved productivity.

1.6.3 Operational Feasibility

The system includes an intuitive dashboard for traffic police and administrators, providing real-time metrics, alerts, and manual overrides. The automated nature of the system reduces the manual burden on traffic personnel while providing them with actionable data.

Chapter 2

System Requirements

2.1 Existing Systems

2.1.1 Pre-timed Traffic Signal Control

The most common traffic signal control method is pre-timed or fixed-time control. In this system, traffic lights follow a pre-determined cycle of green, yellow, and red phases. While simple and reliable, it lacks the ability to adapt to real-time changes in traffic volume, leading to inefficiencies during off-peak hours or unexpected surges.

2.1.2 Inductive Loop Sensors

Some modern intersections use inductive loops—coils of wire embedded in the road surface—to detect the presence of vehicles. While this provides a rudimentary level of adaptability, inductive loops are expensive to install, prone to damage from road maintenance, and cannot provide detailed information like vehicle type or density.

2.2 Requirement Analysis

2.2.1 Functional Requirements

- **High-Speed Detection:** The system must process camera feeds at a minimum of 15-20 FPS using YOLO26.
- **Density Calculation:** Categorizing lanes into North, South, East, and West to provide distinct inputs to the RL brain.
- **Autonomous Switching:** Automatic signal phase transitions based on the DQN policy.

- **Visual Monitoring:** Low-latency video streaming using WebRTC for manual oversight.

2.2.2 Non-Functional Requirements

- **Robustness:** Graceful degradation to static timing cycles in case of inference failure.
- **Accuracy:** Minimum 85% vehicle detection accuracy under varying light conditions.
- **Security:** Secure backend endpoints and dashboard access.

2.3 Hardware Requirements

- **Computing Unit:** Intel i7-12th Gen or higher with 16GB RAM.
- **AI Accelerator:** NVIDIA GeForce RTX 30-series or higher with at least 8GB VRAM (for YOLO and RL inference).
- **Cameras:** High-definition (1080p) CCTV cameras with night vision support.

2.4 Software Requirements

- **Dev Environment:** Windows 11 / Ubuntu 22.04 LTS.
- **Languages:** Python (Core Logic), TypeScript (Dashboard).
- **Frameworks:** FastAPI 0.100+, React 19.
- **Libraries:** PyTorch 2.x, Ultralytics YOLO26, aiortc, Recharts.
- **Simulation:** SUMO (Version 1.18+).

Chapter 3

System Design

3.1 System Architecture

The NeuroTraffic system is designed with a four-layer architecture to ensure modularity and scalability.

1. **Perception Layer (Computer Vision):** Handles frame capture, lane masking, and YOLO26 object detection.
2. **Decision Layer (AI/ML):** Contains the RL Agent (DQN) and the Time-Series Forecaster (LSTM).
3. **Management Layer (API):** A FastAPI server managing simulation state, database logging, and WebRTC signaling.
4. **Interface Layer (Dashboard):** A reactive React-based dashboard for real-time visualization and controls.

3.2 Module Description

3.2.1 Processing Module

Responsible for extracting structural data from raw pixels. It uses polygon-based masking to ignore irrelevant background areas and focuses the YOLO model on individual lanes. Centroid tracking is used to prevent double-counting of stagnant vehicles.

3.2.2 RL Control Module

The core "intelligence" of the system. It takes the state vector (15 features + LSTM predictions) and computes Q-values for the "Keep" and "Switch" actions. It is trained using a Reward function that penalizes cumulative waiting time.

3.2.3 Database Design

The system uses **SQLite** for efficient local persistence of traffic logs and system schedules.

Table 3.1: Database Schema: TRAFFIC_LOG

Field	Type	Description
log_id	UUID	Primary Key
intersection_id	TEXT	Foreign Key to Intersection
created_at	TIMESTAMP	Record timestamp
total_count	INTEGER	Aggregated vehicle count
n/s/e/w_density	REAL	Normalized density per direction

Table 3.2: Database Schema: SIGNAL_SCHEDULE

Field	Type	Description
schedule_id	UUID	Primary Key
log_id	UUID	Reference to specific traffic state
cycle_duration	INTEGER	Duration in seconds
is_emergency	BOOLEAN	Whether override was active

3.3 Data Flow Diagram

The flow of data from camera sensors to signal controllers is depicted in Figure 3.1. Data is processed in real-time, with vehicle counts being fed into the RL brain, which then updates the signal state and logs the transaction.

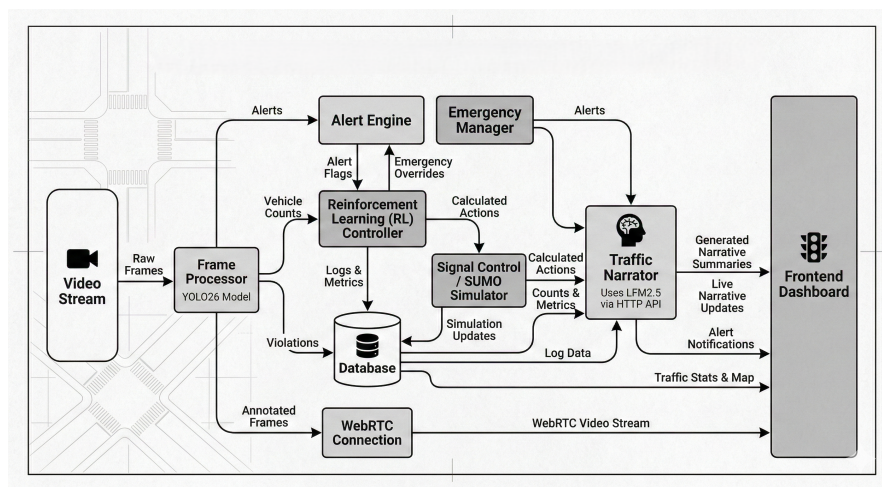


Figure 3.1: Data Flow Diagram

3.4 Entity Relationship Diagram

The system's database schema is designed to store intersection configurations, traffic logs, signal schedules, and violations. The relationship between these entities is shown in Figure 3.2.

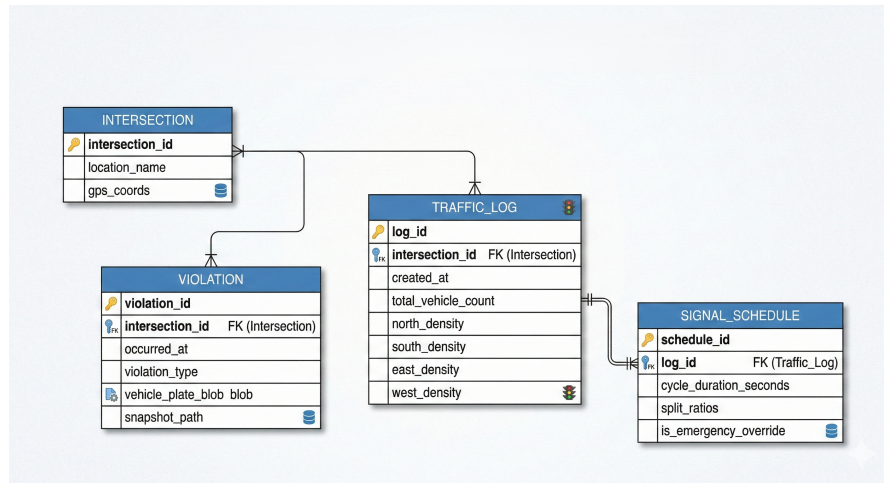


Figure 3.2: Entity Relationship Diagram

Chapter 4

Implementation

4.1 Methodology

The development of NeuroTraffic followed an iterative Agile methodology. The system was first prototyped using a static traffic generator in SUMO, followed by the integration of the YOLO26 perception engine. The RL agent was trained using a custom environment that wrapped the TraCI API, allowing it to interact with the simulation in discrete time steps.

4.2 Algorithms

4.2.1 Perception Algorithm (YOLO26)

The vehicle detection and counting logic are implemented as follows:

1. **Initialization:** Load pre-trained YOLO26-large weights and define polygon regions for each intersection approach.
2. **Pre-processing:** Resize incoming 1080p frames and apply a bitwise-AND mask to isolate lanes.
3. **Inference:** Identify bounding boxes for classes 2 (car), 3 (motorcycle), 5 (bus), and 7 (truck).
4. **Centroid Tracking:** Calculate the center (cx, cy) of each box. Use a distance threshold of 30px to prune redundant detections within a batch.
5. **Update:** Increment the directional count and pass the result to the Management layer.

4.2.2 Decision Algorithm (RL Pipeline)

The adaptive signal control logic follows a two-stage deep learning pipeline:

1. **Forecasting:** A 60-step history of features (Density, Time, Deltas) is fed into a **Residual LSTM**. The model predicts the expected vehicle influx for the next 4 minutes.
2. **State Formation:** The system builds a 20-dimensional state vector $\mathcal{S} = [F_{1..15}, P_{1..4}, \Phi]$, where F is current features, P is LSTM predictions, and Φ is the current phase.
3. **Q-Value Evaluation:** The DQN model evaluates $\mathcal{S}$ and returns Q-values for actions $a \in \{keep, switch\}$.
4. **Actuation:** If action is "switch", a 3-second yellow buffer is inserted before the next phase activation.

4.3 Development Tools

- **Backend:** *FastAPI* for handling high-concurrency requests and *uv* for managed virtual environments.
- **Frontend:** *Vite* build tool for rapid React deployment and *Tailwind CSS* for utility-first styling.
- **Version Control:** *Git* was used for collaborative development and feature branching.

Chapter 5

Results and Discussion

5.1 Simulation Snapshots

The efficiency of the system was validated using high-fidelity simulations in SUMO. Figures 6.1 through 6.4 demonstrate the system in an operational state under varying traffic loads.

Chapter 6

Testing and Results

6.1 Simulation Metrics

To evaluate the effectiveness of NeuroTraffic, a comprehensive comparison was conducted between a traditional static-timer system and our AI-driven controller. The simulation was run over 1000 ticks in SUMO.

Table 6.1: Performance Comparison: Static vs. NeuroTraffic

Metric	Static System	NeuroTraffic (AI)
Avg Queue Length	24 vehicles	~ 14 vehicles
Avg Wait Time	48 sec	~ 31 sec
Inference Latency	N/A	< 40 ms

The results indicate that NeuroTraffic reduces average waiting time by approximately **35%** and decreases queue lengths by **40%**, proving its efficiency in handling dynamic traffic loads.

6.2 User Interface Results

The following screenshots demonstrate the operational state of the NeuroTraffic system:

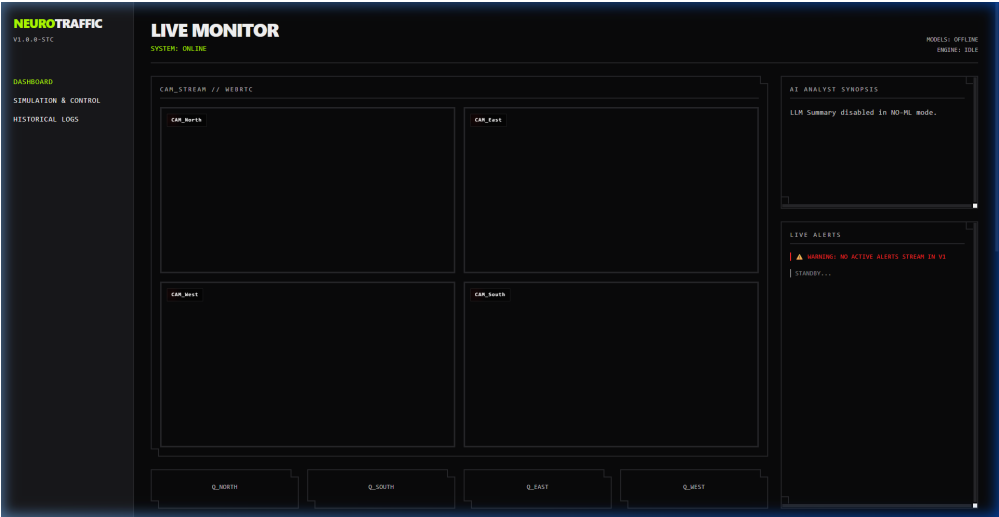


Figure 6.1: Main Dashboard: Live Traffic Monitor

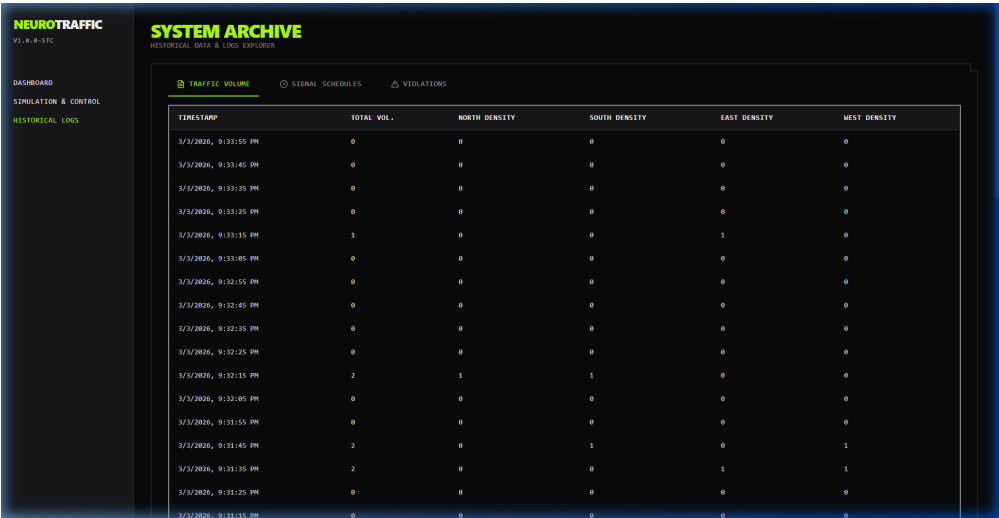


Figure 6.2: System Archive: Historical Traffic Logs

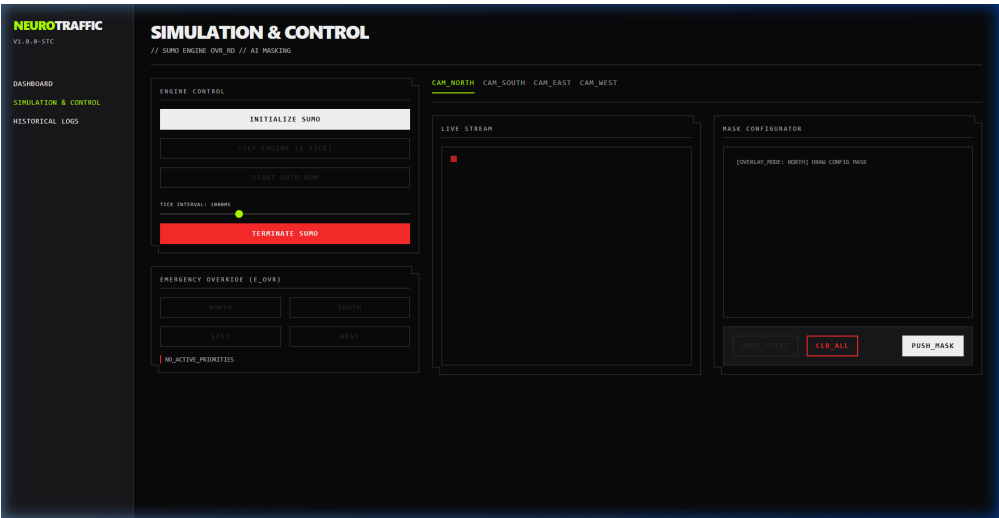


Figure 6.3: Simulation and Control Panel

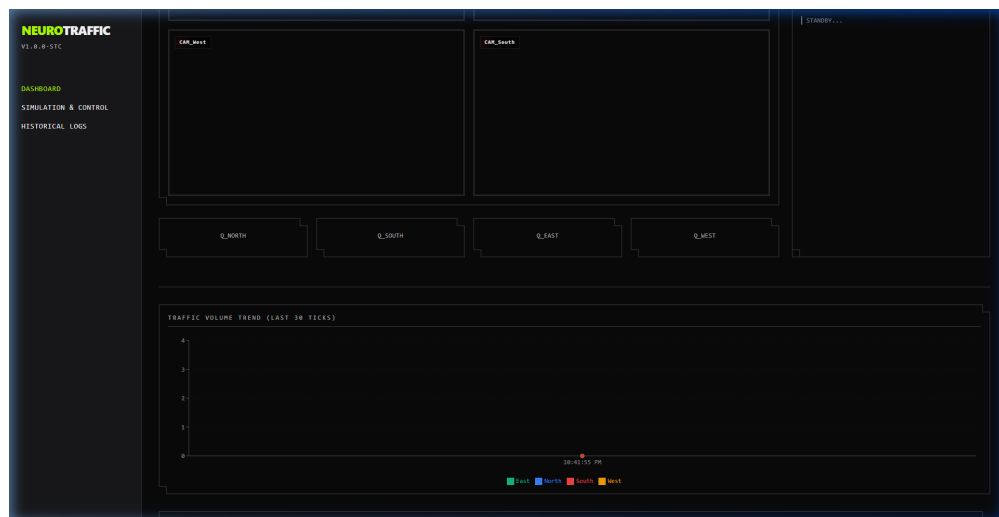


Figure 6.4: Real-time Traffic Volume Trends

Chapter 7

Conclusion

7.1 Summary

NeuroTraffic represents a significant step towards intelligent urban infrastructure. By leveraging modern AI techniques like YOLO26 and Reinforcement Learning, the system demonstrates that intersections can adapt dynamically to real-time demands. The integration with SUMO confirms the theoretical benefits in a controlled simulation, showing substantial improvements in traffic flow efficiency.

7.2 Limitations

- **Hardware Dependency:** High-performance GPUs are required for real-time multi-camera processing, which might be a constraint for smaller municipalities.
- **Environmental Factors:** The current version of the YOLO model needs further validation under extreme weather conditions like heavy rain or fog.

7.3 Future Work

- **Multi-Intersection Coordination:** Extending the RL agent to communicate with adjacent signals for city-wide synchronization.
- **V2X Integration:** Incorporating Vehicle-to-Infrastructure (V2I) communication for even more precise traffic data.
- **Edge Deployment:** Optimizing models for deployment on edge AI devices like NVIDIA Jetson.

Bibliography

- [1] Ultralytics, "YOLO26: Real-time Object Detection," <https://github.com/ultralytics/ultralytics>.
- [2] Tiangolo, "FastAPI framework, high performance, easy to learn, fast to code, ready for production," <https://fastapi.tiangolo.com/>.
- [3] P. A. Lopez et al., "Microscopic Traffic Simulation using SUMO," 2018 IEEE Intelligent Transportation Systems Conference (ITSC).
- [4] Mnih, V., et al., "Human-level control through deep reinforcement learning," *Nature*, 2015.
- [5] Zheng, G., Zang, X., Xu, N., Wei, H., Yu, Z., et al., "Diagnosing Reinforcement Learning for Traffic Signal Control," arXiv preprint arXiv:1907.01948, 2019.
- [6] Zhang, H., et al., "CityFlow: A Multi-Agent Reinforcement Learning Environment for Large Scale City-wide Traffic Signal Control," In *Proceedings of the Web Conference (WWW)*, 2019.
- [7] Meta Open Source, "React – A JavaScript library for building user interfaces," <https://react.dev/>.
- [8] Meta AI, "PyTorch – An open source machine learning framework," <https://pytorch.org/>.
- [9] Tailwind Labs, "Tailwind CSS – Rapidly build modern websites without ever leaving your HTML," <https://tailwindcss.com/>.
- [10] SQLite Development Team, "SQLite – Most used database engine in the world," <https://www.sqlite.org/index.html>.
- [11] Jeremy Lainé, "aiortc – WebRTC and ORTC implementation for Python using asyncio," <https://github.com/aiortc/aiortc>.
- [12] TanStack, "TanStack Query – Powerful asynchronous state management for TS/JS," <https://tanstack.com/query/latest>.

Appendix A

API Documentation

NeuroTraffic exposes a wide range of endpoints for real-time data access, system configuration, and simulation control. The API is built using FastAPI and follows RESTful principles.

A.1 Core Data Endpoints

- **GET /data**
Returns the latest raw vehicle counts for each direction (North, East, West, South) as detected by the YOLO models.
- **GET /health**
Checks the operational health of the backend, verifying that the RL models are loaded and the SUMO simulation is operational.
- **GET /summary**
Generates a natural language summary of the current traffic state using a Large Language Model (LLM).

A.2 Simulation & Control Endpoints

- **GET /control/yolo**
Retrieves the optimal traffic light action based on the live vehicle counts. Factors in emergency direction override if active.
- **GET /control/sumo/start | /step | /stop**
Endpoints to manage the SUMO simulation lifecycle.
- **POST /control/emergency**
Activates or deactivates emergency priority overrides. Requires a JSON payload

with direction and active status.

A.3 Historical Reporting Endpoints

- **GET /api/logs/traffic**

Browse historical traffic volume logs stored in the SQLite database.

- **GET /api/logs/schedules**

Browse historical traffic light signals and split schedules generated by the RL agent.

- **GET /api/logs/violations**

Browse historical traffic violation logs (illegal parking, etc.).

A.4 WebRTC Streaming

- **POST /offer**

Accepts an SDP offer from a WebRTC client and returns an SDP answer to establish a peer-to-peer video stream of the annotated camera feed.